Contents

Interview Prep Pack · Anthropic Final Round

For Holden Ottolini · Solutions Engineering · April 2026

This document has four sections: the opener (tell-me-about-yourself), discovery branches, objection scripts, and a Q&A hot seat. Read it twice before the interview. Don't memorize — internalize.

---

1 · The opener (2 minutes, rehearsed)

Likely question: *"Tell me about yourself and why Anthropic."*

Script (aim for 90–120 seconds):

> "Sure. The short version: I've spent [X years] in [your background] solving problems where the answer was half technical and half human. The moment that brought me to Anthropic was [one specific story — a deployment that almost went wrong, a customer who needed to be told 'no,' or an AI tool you watched a team adopt].
>
> What draws me here specifically, not just to AI broadly, is that Anthropic is the one lab where safety isn't a marketing word. It's in the model's training objective. In a Solutions role, that matters, because customers in regulated industries — exactly the kind of customer I've spent my career serving — don't need a faster tool, they need a trustworthy one. I want to be the person in the room helping them see that Claude is both.
>
> On what I bring: I'm comfortable being the technical person in a non-technical room, and the business person in a technical one. I know when to say 'we can do that' and when to say 'we shouldn't.' That's the SA job."

Rules for delivery: - Start with one sentence of *where you are*, not *where you started*. - Anchor on one specific story, not three generic ones. - End with what *they* get, not what *you want*. - Don't mention the word "passionate." Don't say "journey."

---

2 · Discovery branches (what if they don't give you the answer you expected)

Your five discovery questions are scripted. Their answers won't be. Here's how to pivot without losing the thread.

Q1 — "What does your code review process look like today?"

If they say…	Pivot
"It's fine, no issues"	Press gently: "How long does a typical PR sit before first review?" Most regulated shops are 36+ hours. If they claim <4h, ask about the bottleneck engineer.
"It's the bottleneck"	Lean in hard. Beat 1 of the demo (codebase onboarding) becomes beat 2 (review plugin).
"We don't really review"	Red flag for PCI-DSS. Don't call it out. Just note it and lean into CLAUDE.md as a governance layer.

Q2 — "P1 incidents: harder to find or fix?"

If they say…	Pivot
"Find"	Your strongest lane. Beat 2 of the demo (SRE incident) is your money beat. Spend extra time here.
"Fix"	Reframe: "Once you know the cause, is the fix usually obvious or does it ripple?" This unlocks the "Claude reasons across services" story.
"Both equally painful"	Ideal. Walk through the full SRE beat.

Q3 — "Are engineers using AI tools unofficially?"

If they say…	Pivot
"Yes, Copilot / Cursor / ChatGPT"	Don't trash the competitor. Say: "That's a good signal that adoption won't be the hard part. The harder part is making it auditable."
"No, we've blocked it"	Validate the instinct. Then: "The reason I ask is because shadow adoption is the real risk. Better to have one sanctioned tool with governance than ten unsanctioned ones."

If they say…	Pivot
“We don’t know”	Honest answer. Say: “A common first win in a pilot is just seeing the map. We’ll know after two weeks.”

Q4 — “Biggest security concern?”

If they say…	Pivot
“Data leaving our environment”	Architecture slide (#8). Safety Gate + contractual no-training.
“An engineer committing AI-generated code without review”	CLAUDE.md at scale (#18) + governance hook.
“Hallucinations / wrong answers in production”	Failure modes (#21). “We designed for it, not around it.”
“We haven’t really thought about it yet”	Don’t gloat. Offer: “Let me walk you through what the top three concerns usually are, and you can tell me which map to yours.”

Q5 — “What would success look like 90 days from now?”

If they say…	Pivot
“We’ve rolled out to the whole team”	Gently correct: “That might be aggressive. I’d want you to have a pilot signal first. Let’s define what a good signal looks like.”
“We’ve decided whether to roll out”	Perfect. That’s exactly the framing of your 3-phase plan.
“We’ve shipped [specific outcome]”	Gold. Repeat it back and make it the pilot’s success metric. Write it down.

---

3 · Objection scripts (the big four in regulated fintech)

Each objection has a 20-second script. Use the language exactly — you’ll freestyle less when nervous.

Objection 1 — “We can’t send our code to a third party.”

> “Totally fair. A couple of things. First, source files never leave the laptop in bulk — Claude Code reads locally, and only the session prompt transits. Second, the contract has a no-training clause that’s, frankly, unusual in our industry; I’ll send your legal team the specific language before this week is out. And third, if you still want belt-and-suspenders, we can scope the pilot to

a non-production repo. But I'd gently push back on that last one, because the value shows up faster on real code."

Objection 2 — "Our engineers will become dependent and lose their skills."

"I hear this a lot. Two things. One, Claude plans before it acts, and the engineer sees every proposed change — it's more like code review than autocomplete. The muscle memory it builds is actually 'reading and evaluating code,' which is the skill senior engineers need most. Two, in our customer base, the pattern we see isn't de-skilling — it's senior engineers getting freed from boilerplate and spending more time on architecture. I'd be happy to walk you through a specific case study after the call."

Objection 3 — "We already have Copilot. Why would we add Claude?"

"Copilot is good at one thing: helping an engineer write new code faster in the IDE. Claude Code is designed for a different problem: reasoning across a whole codebase, in a regulated environment, with a human in the loop at every write. For autocomplete, Copilot is fine. For a P1 incident at 2am or an SRE onboarding onto a service they don't own, you want repo-wide reasoning. I'd actually suggest keeping both during the pilot — they solve different problems."

Objection 4 — "What about hallucinations? We can't have AI making things up in production code."

"Correct, and that's exactly why the workflow is what it is. Claude plans before it acts. The Safety Gate shows a full diff, line by line, before any file is touched. The governance hook runs your tests before a commit is accepted. And the engineer's name — not Claude's — is on every commit. The question isn't 'does Claude ever get it wrong?' — the answer to that is yes, sometimes it does. The question is 'what happens when it does?' — and the answer there is: nothing ships."

---

4 · Q&A hot seat (ten questions, prepared answers)

Write these out. Rehearse out loud. Don't wing any of them.

1. "How do you handle it when a customer decides Claude Code isn't right for them?" > "I tell them, early. My job is to help them make a good decision, not a fast one. If after a discovery call it looks like their problem is better solved by an IDE plugin, or that they're not ready for agentic tooling yet, I'll say so. The ones who say no this year are often the ones who say yes next year, and they remember the honesty."

2. "What's a deal you lost, and what did you learn?" > [Prepare a real one. Rule: don't blame the customer, don't blame the product, blame the assumption you made. End with what you do differently now.]

3. “Walk me through how you’d prioritize a week when you have three pilots going and a product gap on one of them.” > “Pilots first, because their timelines are externally promised. The product gap I’d document in a single written brief to the PM with the customer impact, the workaround I’m using, and the urgency. Then I pick one — the one with the closest decision date — and go deep that day, rather than surface across all three.”

4. “How do you decide when to escalate to engineering versus solve it yourself?” > “Three criteria: is it a one-off or a pattern, can the customer wait, and do I know the answer. If I know it and it’s a one-off and they’re blocked, I solve it. Anything else, engineering sees it in writing within the hour.”

5. “A customer asks you to make a technical commitment you’re not sure the product can keep. What do you do?” > “I say ‘I don’t want to commit to that on the call; let me come back tomorrow with a yes or a no.’ Every SA I’ve respected does this. The ones who over-promise are the ones who lose trust.”

6. “Tell me about a time you had to disagree with an engineering decision.” > [Prepare one. Format: what the decision was, why you thought it was wrong, how you raised it, what happened. Bonus if the ending includes ”and I was wrong about part of it.’]

7. “Why you and not someone with more years?” > “Years get you pattern recognition. I have enough pattern recognition to not make the obvious mistakes. What I bring on top is that I’ve been on the customer side of this exact conversation, and I remember what made the difference between a vendor I trusted and one I didn’t. That’s a perspective a career SA doesn’t always have.”

8. “What would you do in your first 30 days?” > “Three things. One, shadow three existing customer calls to calibrate on how our team actually talks to customers — not the script, the reality. Two, pick one industry vertical and go deep — talk to every SA who’s sold into it, read every account note. Three, build one demo from scratch, end to end, that I can deliver solo by day thirty. That’s the deliverable.”

9. “What’s your biggest weakness as an SA?” > [Pick a real one. Not a humble-brag. Something like “I’m slower than I want to be at writing follow-up emails after calls” or “I tend to over-prepare when I should just walk in.” End with what you’re doing about it.]

10. “Any questions for us?”

Always have three ready. Suggested set: - “What’s the hardest kind of customer conversation the SA team runs into right now?” - “How does the SA team get better? Is there a feedback loop from the field back to product?” - “What’s the one thing you wish candidates understood about this role that they usually don’t?”

Don’t ask about comp, benefits, or remote policy in the final round. Save that for the offer call.

---

5 · The last 48 hours

- Rehearse the full 40 minutes out loud, twice, ideally to a friend who’ll interrupt.

- Run the demo on WiFi, on your hotspot, and once with the WiFi intentionally disconnected.
- Pack a charged laptop, a charged phone as hotspot, a second device to take notes, and the PDF of the deck on your phone as a backup.
- Sleep.
- Morning of: light breakfast, no caffeine after 9am, a walk.
- Five minutes before: re-read this document's Q&A section. Nothing else.
- Thirty seconds before: three slow breaths, feet flat on the floor, remind yourself you're not auditioning — you're helping them decide.

Good luck.